

Vibe coding com segurança

12 riscos · código · prompts de validação

Seu app precisa de verificações concretas.

Use este material para entender os riscos citados no vídeo, aplicar controles no código e orientar seu agente a validar o resultado com evidências.

O que você vai revisar

01 Segredos no frontend

02 Entradas sem validação

03 SQL injection

04 Prompt injection

05 Cross-site scripting · XSS

06 IDOR e BOLA

07 SSRF · requisições pelo servidor

08 Senhas em texto puro

09 Abuso de requisições · DoS / DDoS

10 Enumeração e rotas administrativas

11 Bots em login e cadastro

12 Mensagens de erro que vazam dados

Como usar

Leia o risco, adapte o exemplo à sua aplicação e copie o prompt destacado para o agente. Para uma revisão ampla do projeto, use o prompt geral da página 14.

Os exemplos são trechos didáticos em JavaScript/Node.js, com bibliotecas indicadas. Integre-os ao fluxo real de autenticação, autorização e erros. Imports ficam no topo do módulo; variáveis como req, db e session vêm da aplicação. As verificações devem ocorrer em ambiente local ou de teste.

01 Segredos no frontend

O risco

Um arquivo .env separa configuração do código, mas não transforma seu conteúdo em segredo. Se uma chave entra no JavaScript entregue ao navegador, qualquer visitante pode lê-la. No Vite, variáveis com prefixo VITE_ são públicas; chaves privadas devem ficar no servidor.

Como evitar

Mantenha segredos no backend e retorne apenas os dados necessários. Use um gerenciador de segredos no deploy. Se uma chave já vazou, revogue-a e gere outra; apagar o arquivo não desfaz o vazamento.

Node.js · trecho executado somente no servidor

```
// Nunca importar este módulo no frontend.
const apiKey = process.env.SERVICE_API_KEY;
if (!apiKey) throw new Error('Configuração ausente');

const result = await fetch('https://api.example.com/data', {
  headers: { Authorization: `Bearer ${apiKey}` },
  signal: AbortSignal.timeout(3000),
});
if (!result.ok) throw new Error('Falha na integração');
// A rota autenticada devolve só os campos permitidos.
```

A URL example.com é ilustrativa. Integre o trecho a uma rota com autenticação, autorização e tratamento de erros.

PROMPT PARA VALIDAR NO SEU AGENTE

Inspecione as variáveis públicas, o processo de build e os módulos importados pelo frontend. Procure chaves em bundles, mapas de código, respostas HTTP, logs e histórico Git. Use valores fictícios nos testes e mascare qualquer segredo encontrado. Mostre o caminho que levou o valor ao navegador e proponha movê-lo ao backend. Se houve exposição, inclua revogação e rotação; não declare resolvido só por editar o .env.

[Vite · variáveis de ambiente ↗](#)

02 Entradas sem validação

O risco

Campos de formulário, parâmetros de rota, query strings e JSON são dados controlados pelo cliente. A validação no navegador melhora a experiência, mas pode ser ignorada. Aceitar campos inesperados também pode permitir alterações indevidas, como promover uma conta ao receber role: "admin".

Como evitar

Valide formato, tipo, tamanho e regras de negócio no servidor. Aceite só campos previstos e use o resultado validado. Essa etapa complementa as defesas específicas contra SQL injection e XSS.

Node.js + Zod 4 · dentro da rota

```
import { z } from 'zod';
const Profile = z.strictObject({
  name: z.string().trim().min(1).max(80),
  email: z.email(),
});
const parsed = Profile.safeParse(req.body);
if (!parsed.success) {
  return res.status(400).json({ error: 'Dados inválidos' });
}
// Use parsed.data; nunca espalhe req.body no banco.
const input = parsed.data;
```

PROMPT PARA VALIDAR NO SEU AGENTE

Mapeie entradas de body, query, path, headers e uploads. Confira validação no servidor antes do uso e rejeição de campos extras. Em testes locais, envie tipo errado, texto além do limite e role: "admin". Verifique que nenhum dado indevido é persistido e que entradas válidas continuam funcionando. Aponte arquivo, linha, regra ausente e teste de regressão. Não trate validação de formulário como autorização.

[OWASP · validação de entrada ↗](#)

[Zod · schemas ↗](#)

03 SQL injection

O risco

Quando uma entrada é concatenada ao SQL, ela pode alterar a consulta em vez de ser tratada como um valor. Isso abre espaço para leitura ou alteração indevida de dados. O risco existe também em consultas SQL manuais dentro de aplicações que usam um ORM.

Como evitar

Passe valores por parâmetros do driver. Para nomes de coluna, tabela ou ordenação, use uma lista fixa de opções permitidas. Não tente resolver o problema apenas removendo aspas ou palavras suspeitas.

Node.js + PostgreSQL · consulta parametrizada

```
// pool: conexão pg; req.user: identidade já validada.
const { rows } = await pool.query(
  `SELECT id, name FROM projects
   WHERE owner_id = $1 AND name = $2`,
  [req.user.id, input.name]
);
res.json(rows);
```

PROMPT PARA VALIDAR NO SEU AGENTE

Localize SQL manual, chamadas raw e interpolação de strings nas consultas. Diferencie parâmetros de valores e identificadores dinâmicos. Com um banco de teste, confirme que nomes com apóstrofes são tratados como dados e não alteram o filtro de proprietário. Apresente os pontos vulneráveis com evidência, troque concatenação por parâmetros e crie um teste que falhe antes da correção e passe depois.

[OWASP · prevenção de SQL injection ↗](#)

[node-postgres · parâmetros ↗](#)

04 Prompt injection

O risco

Uma pessoa pode esconder instruções em mensagens, páginas, documentos ou resultados de ferramentas que a IA lê. O modelo pode confundir esses dados com ordens e tentar divulgar informações ou executar ações indevidas. Separar instruções e dados ajuda, mas não garante proteção completa.

Como evitar

Trate a saída do modelo como não confiável. Valide cada ferramenta no backend, com permissões da sessão e escopo mínimo. Não entregue segredos desnecessários ao modelo; exija confirmação nas ações de maior impacto.

Node.js + Zod · autorização fora do modelo

```
const Call = z.strictObject({
  tool: z.literal('read_project'),
  projectId: z.string().uuid(),
});
const call = Call.parse(modelOutput);
const project = await db.project.findFirst({
  where: { id: call.projectId, ownerId: session.user.id },
  select: { id: true, name: true },
});
if (!project) throw new Error('Acesso negado');
return project;
```

z, db e session representam o validador, o banco e a sessão autenticada da aplicação.

PROMPT PARA VALIDAR NO SEU AGENTE

Rastreie conteúdo externo até o modelo e até as ferramentas. Em fixtures locais, inclua um documento que peça para ignorar as regras e ler um projeto de outra conta. Confirme que o backend nega a ação mesmo se o modelo a solicitar. Revise permissões, argumentos, dados enviados e aprovação de ações sensíveis. Relate a barreira efetiva e suas limitações; um prompt de sistema sozinho não é evidência de proteção.

[OWASP · prompt injection em LLMs ↗](#)

05 Cross-site scripting · XSS

O risco

XSS acontece quando um dado controlado por outra pessoa vira conteúdo executável no navegador. Um comentário, nome de perfil ou resposta de IA pode injetar HTML ou JavaScript e agir na sessão da vítima. A proteção depende do contexto em que o dado é renderizado.

Como evitar

Para texto comum, use `textContent` ou o escape padrão do framework. Evite `innerHTML` e HTML bruto. Se precisar aceitar HTML rico, use um sanitizador mantido e uma política restritiva; CSP é uma camada adicional.

JavaScript · renderização de texto no DOM

```
const box = document.querySelector('#comment');
const untrustedText = response.comment;

// Mostra o conteúdo como texto, sem interpretá-lo.
box.textContent = untrustedText;

// Não faça: box.innerHTML = untrustedText;
```

PROMPT PARA VALIDAR NO SEU AGENTE

Procure `innerHTML`, `dangerouslySetInnerHTML`, templates sem escape e renderização de Markdown ou HTML retornado por IA. Identifique a origem dos dados e o contexto de saída. Em um teste de navegador isolado, injete um marcador HTML inofensivo e confirme que ele aparece como texto, sem criar elementos ou executar eventos. Se HTML for necessário, revise a sanitização. Mostre evidências e teste de regressão, não apenas a existência de CSP.

[OWASP · prevenção de XSS ↗](#)

06 IDOR e BOLA

O risco

Estar autenticado não autoriza acesso a todos os dados. Em uma falha IDOR, trocar o identificador pode revelar ou alterar um recurso de outra pessoa. Em APIs, BOLA descreve a falha de autorização no nível do objeto. UUIDs dificultam adivinhação, mas não substituem a checagem de permissão.

Como evitar

Filtre o recurso pela identidade autenticada e pelo tenant, quando houver. Aplique a regra em leitura, atualização e exclusão. Não aceite o dono do recurso informado pelo cliente como prova de autorização.

Node.js + Prisma · leitura de recurso próprio

```
// req.user foi preenchido pelo middleware de autenticação.
const project = await prisma.project.findFirst({
  where: {
    id: validatedProjectId,
    ownerId: req.user.id,
  },
  select: { id: true, name: true },
});
if (!project) return res.sendStatus(404);
return res.json(project);
```

PROMPT PARA VALIDAR NO SEU AGENTE

Crie duas contas e dois recursos em um banco local de teste. Com a sessão da conta A, tente ler, editar e apagar o recurso da conta B. Inclua listagens, anexos e limites entre tenants. Espere negação sem vazamento e confirme que A ainda acessa seu próprio recurso. Confira de onde vem a identidade usada no filtro. Relate endpoint, regra aplicada, resposta observada e teste de regressão.

[OWASP · BOLA em APIs ↗](#)

07 SSRF · requisições pelo servidor

O risco

Na SSRF, uma entrada induz seu servidor a fazer uma requisição que ele não deveria fazer. Um importador de URL pode alcançar serviços internos, metadados da nuvem ou destinos externos. Validar apenas o texto do endereço é frágil: redirecionamentos e resolução DNS também importam.

Como evitar

Quando possível, receba uma opção de negócio e escolha o destino no servidor. Evite URLs arbitrárias, bloqueie redirecionamentos e restrinja a saída de rede. Se URLs livres forem necessárias, trate DNS, IPv4, IPv6 e destinos privados.

Node.js · destino fixo escolhido pelo backend

```
const targets = new Map([
  ['catalog', 'https://api.example.com/catalog'],
]);
const target = targets.get(req.query.source);
if (!target) return res.sendStatus(400);
const response = await fetch(target, {
  redirect: 'error',
  signal: AbortSignal.timeout(3000),
});
if (!response.ok) throw new Error('Falha no destino');
```

Substitua example.com por um destino confiável da sua integração. O mapa não dispensa controles de rede e tamanho de resposta.

PROMPT PARA VALIDAR NO SEU AGENTE

Localize fetches, webhooks, importadores e processamento de imagens que aceitam endereços externos. Trace quem controla cada destino. Use mocks locais para testar loopback, IPv6, destinos privados, redirecionamentos e mudança de DNS; não consulte metadados reais da nuvem. Verifique restrição de saída de rede, timeout e limite de resposta. Explique os casos bloqueados e o que depende da infraestrutura, com evidência.

[OWASP · prevenção de SSRF ↗](#)

08 Senhas em texto puro

O risco

Se o banco vazar, senhas em texto puro ficam imediatamente disponíveis para uso. Criptografá-las de forma reversível ainda permite recuperá-las com a chave. Para autenticação, a abordagem usual é armazenar um hash de senha com salt e custo adequado; esse hash não é descriptografado.

Como evitar

Prefira um provedor de autenticação confiável. Se armazenar senhas, use Argon2id por uma biblioteca mantida. Não use SHA-256 puro e não grave senhas em logs, respostas ou ferramentas de análise.

Node.js + argon2 · hash e verificação

```
import argon2 from 'argon2';
const passwordHash = await argon2.hash(password, {
  type: argon2.argon2id,
  memoryCost: 19456, // KiB: 19 MiB
  timeCost: 2,
  parallelism: 1,
});
// Salve passwordHash; a biblioteca gera o salt.
const matches = await argon2.verify(passwordHash, candidate);
// matches é boolean; não existe "descriptografar" o hash.
```

Os parâmetros são um ponto de partida mínimo da OWASP. Meça o custo no seu ambiente e ajuste sem reduzir a proteção.

PROMPT PARA VALIDAR NO SEU AGENTE

Inspecione cadastro, login, recuperação, banco e logs sem revelar senhas. Confirme algoritmo, parâmetros, salt e comparação pela biblioteca. Em teste local, a senha correta deve passar e a incorreta deve falhar; duas contas com a mesma senha devem ter hashes diferentes. Verifique limite de tentativas. Se encontrar texto puro ou criptografia reversível, proponha migração e resposta à exposição, preservando evidências sem copiar credenciais.

[OWASP · armazenamento de senhas ↗](#)

09 Abuso de requisições · DoS / DDoS

O risco

Requisições excessivas ou operações caras podem esgotar CPU, conexões, memória ou orçamento e derrubar a aplicação: negação de serviço, ou DoS. Quando o ataque vem de várias origens, é DDoS. Um rate limit no código reduz abuso, mas sozinho não absorve ataques volumétricos.

Como evitar

Limite por rota e identidade, use armazenamento compartilhado entre instâncias e confie só nos proxies conhecidos. Combine proteção DDoS na borda, WAF, limites de payload e timeout. Uma CDN só ajuda conforme os serviços e regras ativados.

NestJS · JavaScript com suporte a decoradores

```
import { Module } from '@nestjs/common';
import { APP_GUARD } from '@nestjs/core';
import { ThrottlerModule, ThrottlerGuard }
  from '@nestjs/throttler';
@Module({
  imports: [ThrottlerModule.forRoot({
    throttlers: [{ ttl: 60_000, limit: 10 }],
  })],
  providers: [
    { provide: APP_GUARD, useClass: ThrottlerGuard },
  ],
})
export class AppModule {}
```

Exemplo: 10 requisições por 60.000 ms. Ajuste por rota; o armazenamento padrão em memória não coordena várias instâncias.

PROMPT PARA VALIDAR NO SEU AGENTE

Confira guard ativo, unidade do ttl, identidade usada, proxies confiáveis e armazenamento em várias instâncias. Em teste local com limite baixo, confirme HTTP 429 ao exceder a janela e recuperação depois dela. Inspeccione custos de rotas caras e a configuração de proteção da origem. Separe rate limit de proteção DDoS. Não faça teste de carga em produção; documente dependências de infraestrutura e pontos ainda não validados.

[NestJS · rate limiting ↗](#)

[Cloudflare · proteção da origem ↗](#)

10 Enumeração e rotas administrativas

O risco

Enumerar rotas é testar caminhos para descobrir endpoints, painéis e arquivos expostos. Encontrar uma URL não deveria conceder acesso, mas uma rota administrativa sem autorização transforma descoberta em incidente. Ocultar botões, mudar o nome da rota ou bloquear CORS não protege o endpoint.

Como evitar

Exija autenticação e permissão em toda operação sensível. Remova endpoints de depuração, faça inventário de rotas e monitore padrões de acesso. WAF e rate limit ajudam a reduzir varreduras; a autorização continua sendo obrigatória.

Node.js · checagem de permissão no servidor

```
// Middleware executado após autenticar a sessão.
function requireAdmin(req, res, next) {
  if (!req.user) return res.sendStatus(401);
  if (req.user.role !== 'admin') {
    return res.sendStatus(403);
  }
  next();
}
app.get('/admin/reports', authenticate, requireAdmin, handler);
// role vem da sessão validada, nunca do body ou da query.
```

PROMPT PARA VALIDAR NO SEU AGENTE

Faça o inventário a partir do código, sem varrer sites externos. Identifique rotas admin, debug, documentação e arquivos de configuração. Com testes locais, acesse cada rota sensível sem sessão, com usuário comum e com administrador. Confira negação por padrão e permissões vindas de uma fonte confiável. Mostre a matriz de acesso e lacunas; não aceite CORS, URL difícil ou botão escondido como controle de autorização.

[OWASP · autorização ↗](#)

11 Bots em login e cadastro

O risco

Bots automatizam tentativas de senha, criação de contas e outras ações abusivas. Um CAPTCHA exibido na tela não prova que a solicitação passou pelo desafio: o backend precisa validar o token. Mesmo validado, ele é uma camada complementar e pode ser contornado por automação sofisticada.

Como evitar

Combine controles por conta e IP, monitoramento e desafio adaptativo. Valide o token no servidor, incluindo origem e ação esperadas. Em login sensível, use MFA e mensagens que não revelem se uma conta existe.

Node.js · validação no servidor com Turnstile

```
const url = 'https://challenges.cloudflare.com/turnstile/v0/siteverify';
const verdict = await fetch(url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    secret: process.env.TURNSTILE_SECRET,
    response: token,
  }),
  signal: AbortSignal.timeout(3000),
}).then(r => r.ok ? r.json() : null).catch(() => null);
if (!verdict?.success ||
  verdict.hostname !== 'app.example.com' ||
  verdict.action !== 'signup') {
  return res.status(403).json({ error: 'Validação inválida' });
}
```

Valide tipo e tamanho de token antes deste trecho. Ajuste hostname e action ao widget; a chave secreta fica só no servidor.

PROMPT PARA VALIDAR NO SEU AGENTE

Confira a validação do token no backend, o hostname e a action do widget. Teste token ausente, inválido, expirado e reutilizado, além de falha do provedor, usando mocks ou chaves oficiais de teste. Confirme que nenhuma conta é criada nesses casos e que o caso válido passa. Revise limites de tentativas e mensagens de login. Não conclua que há proteção só porque o widget aparece.

[Cloudflare · validação do Turnstile ↗](#)

12 Mensagens de erro que vazam dados

O risco

Uma exceção devolvida diretamente pode expor consultas SQL, caminhos internos, versões, tokens ou dados pessoais. Essas pistas facilitam outros ataques. A resposta ao cliente deve informar o necessário para agir, enquanto a investigação detalhada ocorre em logs protegidos e com dados sensíveis removidos.

Como evitar

Centralize o tratamento de erros inesperados. Retorne mensagem genérica e um identificador de atendimento. Preserve respostas 4xx úteis para erros esperados e não registre corpo, cookies ou Authorization sem uma política de mascaramento.

Express · último middleware de erros

```
import { randomUUID } from 'node:crypto';
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);
  const requestId = randomUUID();
  logger.error({ requestId, event: 'unexpected_error' });
  // Detalhes técnicos: só em logs restritos e sanitizados.
  res.status(500).json({
    error: 'Não foi possível concluir a solicitação.',
    requestId,
  });
});
```

PROMPT PARA VALIDAR NO SEU AGENTE

Force uma exceção controlada em testes locais e inspecione status, corpo e headers da resposta. Procure stack trace, SQL, caminhos, dados pessoais e credenciais. Confira que o requestId permite correlacionar o evento em logs restritos e que os logs não guardam tokens ou senhas. Verifique também 400, 401, 403 e 404 para evitar converter todo erro em 500. Entregue exemplos mascarados e testes de regressão.

[OWASP · tratamento de erros ↗](#)



Valide o projeto com seu agente

Copie o texto abaixo na conversa do agente que tem acesso ao repositório. Informe o ambiente de teste e as regras de acesso da sua aplicação.

CONTEXTO E ESCOPO

Atue como revisor de segurança deste projeto. Identifique a stack, as versões, os pontos de entrada e as fronteiras entre navegador, servidor, banco e serviços externos. Trabalhe primeiro em modo de auditoria. Use somente este repositório e o ambiente local ou de teste autorizado. Não execute carga em produção nem varreduras em serviços externos. Não imprima ou envie segredos.

O QUE INSPECIONAR

Verifique: segredos no frontend e no build; validação de entradas no servidor; SQL parametrizado; prompt injection e autorização das ferramentas de IA; renderização contra XSS; acesso por objeto e tenant (IDOR/BOLA); destinos de requisições contra SSRF; armazenamento de senhas; rate limit, proxies, múltiplas instâncias e proteção DDoS; rotas administrativas; validação de CAPTCHA; respostas de erro e logs.

COMO COMPROVAR

Para cada achado, mostre o fluxo de dados, o arquivo e a linha, a condição necessária, o impacto e uma reprodução mínima com dados fictícios. Diferencie falha confirmada, suspeita e não verificado. Não considere ausência em uma busca textual como prova de segurança. Se faltar acesso à infraestrutura, explique exatamente o que precisa ser conferido.

TESTES E CORREÇÕES

Use os testes existentes e proponha casos negativos e positivos: entrada inválida rejeitada e válida aceita; conta A impedida de acessar B; token de CAPTCHA reutilizado negado; excesso de tentativas limitado. Use mocks para SSRF e provedores externos. Prepare correções mínimas em diff e testes de regressão. Não publique, faça deploy ou altere dados reais como parte da auditoria.

FORMATO DA ENTREGA

Entregue uma tabela com risco, evidência, severidade justificada, correção e teste. Liste comandos executados e resultados reais. Separe bloqueadores de publicação, melhorias e dependências externas. Informe o que não conseguiu testar e o risco residual. Não declare que o aplicativo está seguro apenas porque os testes passaram.



Transforme a revisão em ação

O resultado precisa ser verificável.

Uma revisão útil mostra por que algo falha, onde corrigir e como confirmar a mudança. Use os critérios abaixo para acompanhar a resposta do agente e decidir o próximo passo.

01 Exija evidência

O achado tem arquivo, linha, caminho dos dados e condição de exploração? Uma recomendação genérica não demonstra que aquela falha existe no seu projeto.

02 Confira o teste

O teste reproduz o comportamento indevido e passa após a correção? Há um caso válido que continua funcionando? Um teste que só confere a existência de uma função não basta.

03 Revise a publicação

Confirme controles da infraestrutura, segredos do deploy e comportamento em várias instâncias. Mantenha explícitas as dependências que o agente não conseguiu verificar.

Continue aprendendo comigo.

No canal KipperDev, você encontra conteúdos de programação e tecnologia para entender as decisões por trás do código.

Assistir ao canal KipperDev ↗

youtube.com/@kipperdev

As referências oficiais estão no rodapé de cada tópico. Este guia é um ponto de partida para revisão; a segurança depende também da integração, da infraestrutura e da manutenção contínua.